

HW2: Twitter in a Key-Value Store

Colin Hui - May 2026

Hardware Configuration

I have a MacBook Pro with Apple M1 chip which has 8 cores, 8 GB unified memory, 3.2 GHz, 228 GB SSD (Apple APFS), running macOS 13 Ventura.

Software Stack

Redis 8.6.3 (compiled from source), Python 3.12 (Anaconda), redis-py 7.4.0 for connectivity and for my IDE I used VsCode.

Method Used

I used method 1 where on each postTweet, the tweet is stored and immediately pushed to every follower's home timeline. This makes writes slower but reads fast because timelines are pre-built.

Storage Model

I used these 3 categories of keys:

- followers:{user_id} - This is a Redis set containing the user IDs of everyone who follows this user. It is used during postTweet to know whose timelines to update.
- tweet:{tweet_id} - This is a Redis string that contains a JSON object with tweet_id, user_id, tweet_ts, and tweet_text. Each tweet is stored only once
- timeline:{user_id} - This is a Redis list of tweet keys. For example, "tweet:42" and it is capped at 10 entries using LTRIM. This represents the user's home timeline and it is ready to read with no joins.

Performance Results:	MySQL (HW 1)	Redis (HW 2)
postTweet	2487 calls/sec	2594 calls/sec
getTimeline	314 calls/sec	2032 calls/sec

Analysis

During the process of HW 2, we see that the getTimeline performance improved by almost 6.5x with Redis compared to MySQL (2,032 vs 314 calls/sec). When using MySQL each timeline retrieval needed a JOIN between the follows and tweet tables, an ORDER BY on timestamp, and conversion of results into objects. However, when comparing to Redis, the timeline is pre-built as a list. That means that fetching it is just an LRANGE to get the 10 tweet keys followed by 10 GET calls. This means that joins, sorting or any disk reads are not needed because everything is living in the RAM. Adding on to this. the postTweet rate is actually similar between the two (2594 vs 2487 calls/sec). Redis avoids disk writes and Method 1 adds overhead by pushing each tweet to every follower's timeline at post time. This means that for users

with many followers there would be multiple LPUSH and LTRIM operations per tweet. This would then offset the in-memory speed advantage. So as we can see here the tradeoff is clear where Redis shifts work from read time to write time. On the other hand, MySQL does no extra work on insert but is more expensive on every timeline read. Redis does more work on insert but makes reads basically free. Now in my opinion for a platform like Twitter where timelines are read far more often than tweets are posted I feel that this is the right tradeoff. However we see that both systems that I used for the homeworks are still extremely short of Twitter's actual demand of 200,000–300,000 timeline reads/sec. Achieving that scale would probably require Method 2 with the pub/sub with background workers) as well as possibly Redis clustering across multiple servers.

AI Usage

I did not use the help of AI for this homework - Sorry it is slightly than 1 page and an hour late